Princeton University

ORF 363/COS 323
Final Exam, Spring 2023

May 11–13, 2023

Instructor:
I. Akrotirianakis

AIs:
Z. Li, R. Sambharya, W. Underwood

1. Please write your name on the first page of your solutions. Next to it, please write out and sign the following pledge: "I pledge my honor that I have not violated the Honor Code or the rules specified by the instructor during this examination."
2. The exam is not to be discussed with *anyone* except possibly the instructors and the AIs. You can only ask *clarification questions*, and only as *public* (and preferably non-anonymous) questions on Ed Discussion. No emails.
3. You are allowed to consult the lecture notes, your own notes, the reference books of the course as indicated on the syllabus, the problem sets and their solutions (yours and ours), the midterm and its solutions (yours and ours), the practice midterm and final exams and their solutions, and all Ed Discussion posts, but *nothing else*. You can only use the Internet in case you run into problems related to software.
4. You may refer to facts proven in the notes or problem sets without reproving them.
5. For all computational problems, include your code. The output that you present should come from your code.
6. Unless you have been granted an extension because of overlapping finals, the exam is to be turned in on Saturday (May 13, 2023) at 10 AM EDT on Gradescope as a single PDF file. You are free to write your solutions on paper or on a tablet, or to type them up. Only the latest version submitted before your deadline will be graded.
7. Each question has 25 points. You need to justify your answers to receive full credit.

Problem 1: Convex sets, convex functions and optimality conditions

a) Consider the two sets

$$S_1 = \left\{(x_1, x_2) \in \mathbb{R}^2 \mid x_1 + 2x_2 - 1 \geq 0,\ 2x_1 + x_2 - 1 \leq 0\right\},$$
$$S_2 = \left\{(x_1, x_2) \in \mathbb{R}^2 \mid x_1 + 2x_2 - 1 \leq 0,\ 2x_1 + x_2 - 1 \geq 0\right\}.$$

Is the set $S = S_1 \cap S_2$ a convex set? Fully justify your answer.
Is the set $S' = S_1 \cup S_2$ a convex set? Fully justify your answer.

b) Find the range of values of the parameter $\alpha \in \mathbb{R}$ for which the function

$$f(x_1, x_2, x_3) = 4x_1^2 + x_2^2 + 5x_3^2 + 2\alpha x_1 x_2 - 2x_1 x_3 + 4\alpha x_2 x_3$$

is convex.

c) Show that $x^* = \left(1, 1/2, -1\right)^\mathsf{T}$ is optimal for the constrained optimization problem

$$\begin{aligned} \min_{x \in \mathbb{R}^3} \quad & f(x) = \tfrac{1}{2} x^\mathsf{T} Q x + b^\mathsf{T} x + c \\ \text{s.t.} \quad & -1 \leq x_i \leq 1, \quad i \in \{1, 2, 3\}, \end{aligned} \tag{1}$$

where

$$Q = \begin{pmatrix} 13 & 12 & -2 \\ 12 & 17 & 6 \\ -2 & 6 & 12 \end{pmatrix}, \quad b = \begin{pmatrix} -22 \\ -14.5 \\ 13 \end{pmatrix} \quad \text{and} \quad c = 1.$$

Solution

a) The two sets S_1 and S_2 are convex because they are defined by affine inequalities. The set S is an intersection of two convex sets and hence it is convex.

The set S' is not convex. To see this, note that $(0,1)^\mathsf{T} \in S_1$ and $(1,0)^\mathsf{T} \in S_2$ but the convex combination $\frac{1}{2}(0,1)^\mathsf{T} + \frac{1}{2}(1,0)^\mathsf{T} = (1/2, 1/2)^\mathsf{T} \notin S_1 \cup S_2$.

b) Since f is quadratic we find its Hessian and determine the values of α that make it PSD using Sylvester's criterion. The first-order partial derivatives of f are

$$\frac{\partial f}{\partial x_1} = 8x_1 + 2\alpha x_2 - 2x_3, \quad \frac{\partial f}{\partial x_2} = 2x_2 + 2\alpha x_1 + 4\alpha x_3, \quad \frac{\partial f}{\partial x_3} = 10x_3 - 2x_1 + 4\alpha x_2.$$

The second-order partial derivatives are

$$\frac{\partial^2 f}{\partial x_1^2} = 8, \quad \frac{\partial^2 f}{\partial x_1 \partial x_2} = 2\alpha, \quad \frac{\partial^2 f}{\partial x_1 \partial x_3} = -2, \quad \frac{\partial^2 f}{\partial x_2^2} = 2, \quad \frac{\partial^2 f}{\partial x_2 \partial x_3} = 4\alpha, \quad \frac{\partial^2 f}{\partial x_3^2} = 10.$$

Hence the Hessian of f is

$$H = \begin{pmatrix} 8 & 2\alpha & -2 \\ 2\alpha & 2 & 4\alpha \\ -2 & 4\alpha & 10 \end{pmatrix} = 2 \begin{pmatrix} 4 & \alpha & -1 \\ \alpha & 1 & 2\alpha \\ -1 & 2\alpha & 5 \end{pmatrix}.$$

We apply Sylvester's criterion, finding all of the principal minors of H. Note that we can ignore the factor of 2. It is easy to see that the 1×1 principal minors are all positive. Next we check the 2×2 principal minors:

$$\det(H_{12}) = \det \begin{pmatrix} 4 & \alpha \\ \alpha & 1 \end{pmatrix} = 4 - a^2 \geq 0 \iff \alpha \in [-2, 2], \tag{2}$$

$$\det(H_{13}) = \det \begin{pmatrix} 4 & -1 \\ -1 & 5 \end{pmatrix} = 19 \geq 0 \iff \alpha \in \mathbb{R}, \tag{3}$$

$$\det(H_{23}) = \det \begin{pmatrix} 1 & 2\alpha \\ 2\alpha & 5 \end{pmatrix} = 5 - 4a^2 \geq 0 \iff \alpha \in \left[-\sqrt{5}/2, \sqrt{5}/2\right]. \tag{4}$$

Finally we consider the 3×3 principal minor, which is just H:

$$\det(H) = \det \begin{pmatrix} 4 & \alpha & -1 \\ \alpha & 1 & 2\alpha \\ -1 & 2\alpha & 5 \end{pmatrix} \tag{5}$$

$$
\begin{aligned}
&= 4\det\begin{pmatrix} 1 & 2\alpha \\ 2\alpha & 5 \end{pmatrix} - \alpha\det\begin{pmatrix} \alpha & 2\alpha \\ -1 & 5 \end{pmatrix} + (-1)\det\begin{pmatrix} \alpha & 1 \\ -1 & 2\alpha \end{pmatrix} \\
&= 4(5-4\alpha^2) - \alpha(5\alpha+2\alpha) - (2\alpha^2+1) \\
&= 19 - 25\alpha^2 \geq 0 \quad \Longleftrightarrow \quad \alpha \in \left[-\sqrt{19}/5, \sqrt{19}/5\right]. \qquad (6)
\end{aligned}
$$

Taking the intersection of the ranges described by (2-6) we can deduce that f is convex if and only if $\alpha \in \left[-\sqrt{19}/5, \sqrt{19}/5\right]$.

c) The objective function is convex because its Hessian Q is positive definite. The constraints are affine inequalities, so this is a convex optimization problem.

For a convex optimization problem a feasible point $x^* \in \Omega$ is optimal if and only if $\nabla f(x^*)^\mathsf{T}(y - x^*) \geq 0 \ \forall y \in \Omega$. The point $x^* = (1, 1/2, -1)^\mathsf{T}$ is feasible and we have $\nabla f(x^*) = (-1, 0, 2)^\mathsf{T}$. Also $\nabla f(x^*)^\mathsf{T}(y - x^*) = -(y_1 - 1) + 2(y_2 + 1) \geq 0$ since $-1 \leq y_i \leq 1$. Therefore x^* is optimal for (1).

Problem 2: Semidefinite programming

Consider the Petersen graph $G = (V, E)$ given in Figure 1.

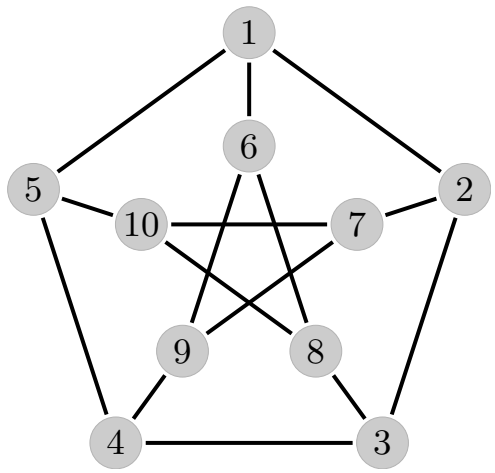

Figure 1: The Petersen graph with 10 nodes.

The stable number of the Petersen graph is $\alpha(G) = 4$.

a) Write down the vertex set V, the edge set E and the adjacency matrix A for the Petersen graph.

b) Formulate the maximum stable set problem as an integer programming problem. Solve its linear programming (LP) relaxation using CVX or CVXPY and report its optimal value. What is the relationship between this value and $\alpha(G)$?

c) Formulate the Lovász semidefinite program (SDP) relaxation, solve it using CVX or CVXPY, and report its optimal value. What is the relationship between this value and $\alpha(G)$?

d) What can you say about the optimal solutions of the LP and SDP relaxations? Which one is of better quality and why?

Solution

a) The Petersen graph is defined as $G = (V, E)$ where $V = \{1, \ldots, 10\}$ is the set of vertices and $E = \{\{1,2\}, \{2,3\}, \{3,4\}, \{4,5\}, \{5,1\}, \{1,6\}, \{2,7\}, \{3,8\}, \{4,9\}, \{5,10\}, \{6,8\}, \{7,9\}, \{8,10\}, \{9,6\}, \{10,7\}\}$ is the set of edges. We have $n = 10$ vertices and 15 edges. The adjacency matrix is $A \in \mathbb{R}^{n \times n}$ with $A_{ij} = 1$ if $\{i,j\} \in E$ and zero otherwise. The diagonal elements of A are all zero since there are no edges from a vertex to itself:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \end{pmatrix}.$$

b) A stable set S is a subset of V containing no adjacent vertices in the graph G. The maximum stable set problem is defined as follows: for a given graph $G = (V, E)$ find a stable set that has the maximum cardinality. We denote such a set by $S_{\max}$ and its cardinality $|S_{\max}|$ is called the stability number, $\alpha(G)$, of the graph.

We can formulate the maximum stable set as an integer programming problem. Define the binary decision variables $x_i \in \{0,1\}$ for $1 \leq i \leq n$ by $x_i = 1$ if $v_i \in S_{\max}$ and zero otherwise. Our objective is to maximize $f(x) = \sum_{i=1}^{n} x_i$. We must ensure that no two adjacent vertices are included in the maximal stable set. This requirement can be expressed by the constraints $x_i + x_j \leq 1$ for all edges $(i,j) \in E$. Hence the IP formulation of the maximum stable set is described by the following optimization problem:

$$\begin{aligned} \max \quad & \sum_{i=1}^{n} x_i \\ \text{s.t.} \quad & x_i + x_j \leq 1, \quad (i,j) \in E, \\ & x_i \in \{0,1\}, \quad 1 \leq i \leq n. \end{aligned} \tag{7}$$

The LP relaxation of problem (7) is defined by allowing the binary decision variables to take continuous values in the interval $[0, 1]$:

$$\begin{aligned} \max \quad & \sum_{i=1}^{n} x_i \\ \text{s.t.} \quad & x_i + x_j \leq 1, \quad (i,j) \in E, \\ & x_i \in [0,1], \quad 1 \leq i \leq n. \end{aligned} \tag{8}$$

The optimal value of this problem is 5. The LP-relaxation (8) in Matlab using CVX is described below.

```
Adj = [0 1 0 0 1 1 0 0 0 0;
       1 0 1 0 0 0 1 0 0 0;
       0 1 0 1 0 0 0 1 0 0;
       0 0 1 0 1 0 0 0 1 0;
       1 0 0 1 0 0 0 0 0 1;
       1 0 0 0 0 0 0 1 1 0;
       0 1 0 0 0 0 0 0 1 1;
       0 0 1 0 0 1 0 0 0 1;
       0 0 0 1 0 1 1 0 0 0;
       0 0 0 0 1 0 1 1 0 0];

% LP relaxation
[~,N] = size(Adj)
cvx_begin
    variable xx(N);
    maximize sum(xx)
    subject to
        for i = 1:N-1
            for j=i+1:N
                if Adj(i,j) == 1
                    xx(i) + xx(j) <= 1;
                end
            end
        end
        xx>=0;
        xx<=1;
cvx_end
```

The LP-relaxation (8) in Python using CVXPY is described below.

```
import numpy as np
```

```
import cvxpy as cp

A = np.array([[0, 1, 0, 0, 1, 1, 0, 0, 0, 0],
              [1, 0, 1, 0, 0, 0, 1, 0, 0, 0],
              [0, 1, 0, 1, 0, 0, 0, 1, 0, 0],
              [0, 0, 1, 0, 1, 0, 0, 0, 1, 0],
              [1, 0, 0, 1, 0, 0, 0, 0, 0, 1],
              [1, 0, 0, 0, 0, 0, 0, 1, 1, 0],
              [0, 1, 0, 0, 0, 0, 0, 0, 1, 1],
              [0, 0, 1, 0, 0, 1, 0, 0, 0, 1],
              [0, 0, 0, 1, 0, 1, 1, 0, 0, 0],
              [0, 0, 0, 0, 1, 0, 1, 1, 0, 0]])

n = len(A)
x = cp.Variable(n)
constrLP = [0 <= x, x <= 1]

for i in range(n):
    for j in range(n):
        if A[i, j] == 1:
            constrLP += [x[i] + x[j] <= 1]

objLP = cp.sum(x)
probLP = cp.Problem(cp.Maximize(objLP), constrLP)
probLP.solve()
print(probLP.value)
```

c) The Lovasz SDP relaxation of the maximum stable set problem is described by the following optimization problem.

$$\begin{aligned} \max \quad & \mathrm{Tr}(JX) \\ \text{s.t.} \quad & \mathrm{Tr}(X) = 1 \\ & X_{ij} = 0, \forall (i,j) \in E \\ & X \succeq 0 \end{aligned} \tag{9}$$

The optimal value of (9) is 4. The SDP-relaxation (9) in **Matlab** using **CVX** is described below.

```
Adj = [0 1 0 0 1 1 0 0 0 0;
       1 0 1 0 0 0 1 0 0 0;
       0 1 0 1 0 0 0 1 0 0;
```

```
        0 0 1 0 1 0 0 0 1 0;
        1 0 0 1 0 0 0 0 0 1;
        1 0 0 0 0 0 0 1 1 0;
        0 1 0 0 0 0 0 0 1 1;
        0 0 1 0 0 1 0 0 0 1;
        0 0 0 1 0 1 1 0 0 0;
        0 0 0 0 1 0 1 1 0 0];

% Lovasz SDP
[~,n] = size(Adj)
J=ones(n);
cvx_begin
    variable X(n,n) symmetric;
    variable x(n);
    maximize trace(J*X)
    subject to
        trace(X) == 1;
        X == semidefinite(n);
        X(Adj == 1) == 0;
cvx_end
```

The SDP-relaxation (9) in **Python** using **CVXPY** is described below.

```
import numpy as np
import cvxpy as cp

# define the adjacency matrix
A = np.array([[0, 1, 0, 0, 1, 1, 0, 0, 0, 0],
              [1, 0, 1, 0, 0, 0, 1, 0, 0, 0],
              [0, 1, 0, 1, 0, 0, 0, 1, 0, 0],
              [0, 0, 1, 0, 1, 0, 0, 0, 1, 0],
              [1, 0, 0, 1, 0, 0, 0, 0, 0, 1],
              [1, 0, 0, 0, 0, 0, 0, 1, 1, 0],
              [0, 1, 0, 0, 0, 0, 0, 0, 1, 1],
              [0, 0, 1, 0, 0, 1, 0, 0, 0, 1],
              [0, 0, 0, 1, 0, 1, 1, 0, 0, 0],
              [0, 0, 0, 0, 1, 0, 1, 1, 0, 0]])

n = len(A)

# define the Lovasz SDP relaxation
X = cp.Variable((n,n), PSD=True)
constrSDP = [cp.trace(X) == 1]
```

```
for i in range(n):
    for j in range(n):
        if i < j:
            if A[i,j] == 1:
                constrSDP += [X[i,j] == 0]

J = np.ones((n, n))
objSDP = cp.trace(J @ X)
probSDP = cp.Problem(cp.Maximize(objSDP), constrSDP)
probSDP.solve()
print(probSDP.value)
```

d) It was shown in class that $\alpha(G) \leq \mathrm{SDP}_{\mathrm{OPT}} \leq \mathrm{LP}_{\mathrm{OPT}}$, so the SDP gives a better upper bound. In fact since $\mathrm{SDP}_{\mathrm{OPT}} = \alpha(G) = 4$, the SDP upper bound is tight.

Problem 3: Reunions planning

You have been tasked with the problem of creating an optimal schedule for preparing for Princeton Reunions 2023. There are several tasks which need to be completed, and each will take a different amount of time. Further, some tasks cannot be started until other tasks have been completed. Reunions must open on May 25, 2023, which is day 145 in the year. A summary of the tasks (your problem data) is given in Table 1.

Task	Description	Duration	After
`tents`	Build the tents	23 days	none
`signs`	Put up signposts	8 days	`fences`
`lighting`	Set up lighting on campus	7 days	`generators`, `signs`, `tents`
`fireworks`	Prepare the fireworks show	6 days	`generators`
`stages`	Build the music stages	9 days	`tents`, `generators`
`generators`	Install electrical generators	4 days	`tents`
`fences`	Build fences around areas	16 days	none
`food`	Deliver the food	1 day	`tables`
`tables`	Put up dining tables	3 days	`generators`, `tents`
`opening`	Reunions opening!	1 day	all other tasks

Table 1: Summary of the tasks

For example, if we begin putting up `tables` on day 138 (May 18), we cannot deliver the `food` until day 141 (May 21) at the earliest. We can begin working on the `tents` at any time, since they do not depend on any other tasks. All of the tasks must be completed before Reunions `opening`. In order to cause minimum disruption on campus, your objective is to begin the first task on the latest day possible.

a) Show that the optimization problem described above can be expressed as a linear program. You may wish to consider the decision variable t_i for $1 \leq i \leq 10$ which represents the day on which we should begin task i.

b) Solve the resulting linear program using CVX or CVXPY. What day of the year should we begin working on the first task? What date does this correspond to?

c) Is the optimal solution unique? If so, provide a proof. If not, present a counterexample. You may wish to consider maximizing $t_i - t'_i$ for an appropriate choice of i, where t and t' are both optimal solutions to the original problem.

Solution

a) Let $n = 10$ be the number of tasks. For $1 \leq i \leq n$ let $t_i \in \mathbb{R}$ be the day on which we begin task i, let $d_i \in \mathbb{R}$ be the duration in days of task i, and let $A_i \subseteq \{1 \ldots, n\}$ be the indices of the tasks which come before task i. Let $s \in \mathbb{R}$ be the day on which we start the first task. We formulate the following linear program.

$$\begin{array}{ll} \text{maximize} & s \\ \text{subject to} & t_n = 145, \\ & t_i \geq t_j + d_j, \qquad \forall j \in A_i,\ \forall i \in \{1, \ldots, n\}, \\ & t_i \geq s, \qquad \forall i \in \{1, \ldots, n\}. \end{array}$$

b) The optimal value of this linear program is 109, so we should start the first task on day 109 (April 19).

c) To determine the uniqueness of the solution, let t_j and t'_j represent two different solutions and consider the following linear program.

$$\begin{array}{ll} \text{maximize} & t_2 - t'_2 \\ \text{subject to} & t_n = 145, \\ & t'_n = 145, \\ & t_i \geq t_j + d_j, \qquad \forall j \in A_i,\ \forall i \in \{1, \ldots, n\}, \\ & t'_i \geq t'_j + d_j, \qquad \forall j \in A_i,\ \forall i \in \{1, \ldots, n\}, \\ & t_i \geq s, \qquad \forall i \in \{1, \ldots, n\}, \\ & t'_i \geq s', \qquad \forall i \in \{1, \ldots, n\}, \\ & s = 109, \\ & s' = 109. \end{array}$$

Note that this linear program has a non-zero optimal value if and only if t and t' are optimal solutions to the original problem with $t_2 \neq t'_2$. Solving this in CVX or CVXPY reveals that the optimal value is 5. Two such optimal solutions are given in Table 2.

Solution 1	Solution 2
`tents` on day 109	`tents` on day 109
`signs` on day 130	`signs` on day 125
`lighting` on day 138	`lighting` on day 138
`fireworks` on day 139	`fireworks` on day 139
`stages` on day 136	`stages` on day 136
`generators` on day 132	`generators` on day 132
`fences` on day 109	`fences` on day 109
`food` on day 144	`food` on day 144
`tables` on day 141	`tables` on day 141
`opening` on day 145	`opening` on day 145

Table 2: Two different optimal solutions for the optimization problem.

Here is the **Python** code:

```
import cvxpy as cp

names = ["tents", "signs", "fences", "generators", "fireworks",
         "stages", "lighting", "tables", "food", "opening"]

durations_dict = {"tents": 23, "signs": 8, "fences": 16,
                  "generators": 4, "fireworks": 6, "stages": 9,
                  "lighting": 7, "tables": 3, "food": 1,
                  "opening": 0}

afters_dict = {"tents": [], "signs": ["fences"], "fences": [],
               "generators": ["tents"], "fireworks": ["generators"],
               "stages": ["tents", "generators"],
               "lighting": ["generators", "signs", "tents"],
               "tables": ["generators", "tents"], "food": ["tables"],
               "opening": ["tents", "signs", "fences", "generators",
                           "fireworks", "stages", "lighting",
                           "tables", "food"]}

n = len(names)
afters = [[j for j in range(n) if names[j] in afters_dict[name]]
          for name in names]
durations = [durations_dict[name] for name in names]
```

```
# solve main problem
t = cp.Variable(n)
s = cp.Variable()
objective = cp.Maximize(s)
constraints = [t[-1] == 145]

for i in range(n):
    constraints += [t[i] >= t[j] + durations[j] for j in afters[i]]
    constraints += [t[i] >= s]

prob = cp.Problem(objective, constraints)
result = prob.solve()

print("The optimal schedule starts on day", round(result))

# find two alternate optimal schedules
t1 = cp.Variable(n)
t2 = cp.Variable(n)
s1 = cp.Variable()
s2 = cp.Variable()
objective = cp.Maximize(t1[1] - t2[1])
constraints = [t1[-1] == 145, t2[-1] == 145]
constraints += [s1 == s.value, s2 == s.value]

for i in range(n):
    constraints += [t1[i] >= t1[j] + durations[j] for j in afters[i]]
    constraints += [t2[i] >= t2[j] + durations[j] for j in afters[i]]
    constraints += [t1[i] >= s1]
    constraints += [t2[i] >= s2]

prob = cp.Problem(objective, constraints)
result = prob.solve()

print("\nOne optimal schedule is")
for i in range(n):
    print("Do", names[i], "on day", round(t1.value[i]))

print("\nAnother optimal schedule is")
for i in range(n):
    print("Do", names[i], "on day", round(t2.value[i]))
```

Here is the **Matlab** code:

```
names = ["tents", "signs", "fences", "generators", "fireworks", "stages",
```

```
    "lighting", "tables", "food", "opening"];

durations_dict = dictionary("tents", 23, "signs", 8, "fences", 16, "
    generators", 4, "fireworks", 6, "stages", 9, "lighting", 7, "tables",
    3, "food", 1, "opening", 0);

afters_dict = dictionary("tents", {[]}, "signs", {["fences"]}, "fences",
    {[]}, "generators", {["tents"]}, "fireworks", {["generators"]}, "stages
    ", {["tents", "generators"]}, "lighting", {["generators", "signs", "
    tents"]}, "tables", {["generators", "tents"]}, "food", {["tables"]}, "
    opening", {["tents", "signs", "fences", "generators", "fireworks", "
    stages", "lighting", "tables", "food"]});

n = length(names);
afters = zeros(n, n);

for i = 1:n
    name = names(i)
    for j = 1:n
        for k = afters_dict{name}
            if names(j) == k
                afters(i, j) = 1;
            end
        end
    end
end

durations = []
for name = names
    durations = [durations, durations_dict(name)]
end

% solve main problem
cvx_begin
variable t(n)
variable s
maximize(s)
subject to

t(end) == 145

for i = 1:n
```

```
    t(i) >= s
    for j = 1:n
        if afters(i,j) == 1
            t(i) >= t(j) + durations(j)
        end
    end
end

cvx_end

display("The optimal schedule starts on day")
display(cvx_optval)

%# find two alternate optimal schedules
cvx_begin
variable t1(n)
variable t2(n)
variable s1
variable s2
maximize(t1(2) - t2(2))
subject to

t1(end) == 145
t2(end) == 145
s1 == 109
s2 == 109

for i = 1:n
    t1(i) >= s
    t2(i) >= s
    for j = 1:n
        if afters(i,j) == 1
            t1(i) >= t1(j) + durations(j)
            t2(i) >= t2(j) + durations(j)
        end
    end
end

cvx_end

display("One optimal schedule is")
for i =1:n
```

```
    display(names(i))
    display(t1(i))
end

display("Another optimal schedule is")
for i =1:n
    display(names(i))
    display(t2(i))
end
```

Problem 4: Data-fitting

Consider the data-fitting problem where we are given N data points consisting of inputs $\{x_i\}_{i=1}^N$ and corresponding outputs $\{y_i\}_{i=1}^N$. The goal is to find a function $f : \mathbb{R} \to \mathbb{R}$ so that $f(x_i) \approx y_i$. Assume that the function f takes the form

$$f(x) = c_0 + c_1 x + c_2 x^2 + c_3 x^3,$$

where the coefficients $c = (c_0, c_1, c_2, c_3)$ are unknown. We aim to find the vector c that minimizes

$$\frac{1}{2}\sum_{i=1}^{N}(f(x_i) - y_i)^2. \tag{10}$$

a) Show that problem (10) can be written as the least squares problem

$$\text{minimize} \qquad g(c) = \frac{1}{2}\|Ac - b\|_2^2,$$

where A and b are given as problem data and c is the decision variable. Specify how A and b are formed and what their dimensions are.

b) Now, for the rest of the problem we will deal with real data points. The data points can be generated in Matlab with

```
x = linspace(-2, 2, 15)
y = [-8.64, -3.11, 0.23, -0.14, 2.80, 2.09, -0.47, -0.27, -1.05,
     -4.33, -6.19, -8.58, -11.46, -10.48, -8.89].
```

If you prefer Python they can be generated with

```
import numpy as np
x = np.linspace(-2, 2, 15)
y = np.array([-8.64, -3.11, 0.23, -0.14, 2.80, 2.09, -0.47, -0.27,
              -1.05, -4.33, -6.19, -8.58, -11.46, -10.48, -8.89]).
```

Implement gradient descent with a constant step size of $\alpha = 0.004$. Run gradient descent starting at $c^0 = (0, 0, 0, 0)$ for 1000 iterations and plot the gradient with a log scale on the y-axis. Report the optimal solution obtained from gradient descent.

c) We want to analyze Newton's method to solve this least squares problem. Write an expression for the first update, c^1, and prove that it is optimal regardless of the initial point, c^0. Make your argument based on the optimality conditions of this problem. Verify that the solution obtained from Newton's method is that obtained in part (b),

d) Plot the fitted cubic function $f(x)$ as a curve, and show the data points as a scatter plot on the same axes.

Solution

1. The vector $b \in \mathbb{R}^N$ is formed as $b = (y_1, \ldots, y_N)^\intercal$. The matrix $A \in \mathbb{R}^{N \times 4}$ is formed as

$$A = \begin{bmatrix} 1 & x_1 & x_1^2 & x_1^3 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_N & x_N^2 & x_N^3 \end{bmatrix}. \tag{11}$$

   The result is that $Ac = (f(x_1), \ldots, f(x_N))^\intercal$.

2. The gradient of the objective is $\nabla g(c) = A^T(Ac - b)$. The steepest descent algorithm follows the iterations

$$c^{i+1} = c^i - \alpha \nabla g(c). \tag{12}$$

   Figure (2) shows linear convergence to the optimal solution. The optimal solution is

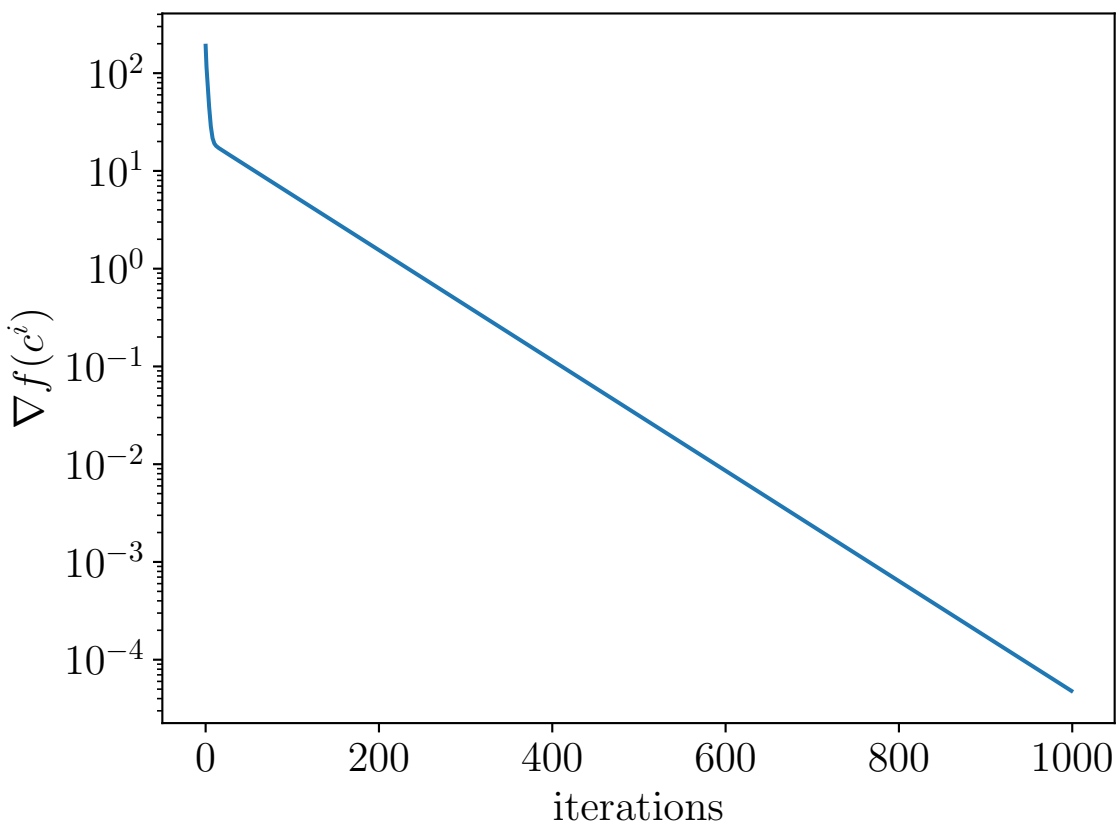

Figure 2: Gradient descent convergence.

   $c = (-0.652, -6.294, -2.131, 1.490)$.

3. The Hessian is $\nabla^2 g(c) = A^T A$ which is invertible since the columns of A are linearly independent. Newton's method follows the iterations

$$c^{i+1} = c^i - \left(\nabla^2 g(c)\right)^{-1} \nabla g(c). \tag{13}$$

Simplifying this expression yields the result

$$\begin{aligned} c^{i+1} &= c^i - (A^T A)^{-1}(A^T A c^i - A^T b) \\ &= c^i - c^i + (A^T A)^{-1} A^T b \\ &= (A^T A)^{-1} A^T b. \end{aligned}$$

4. See figure (3).

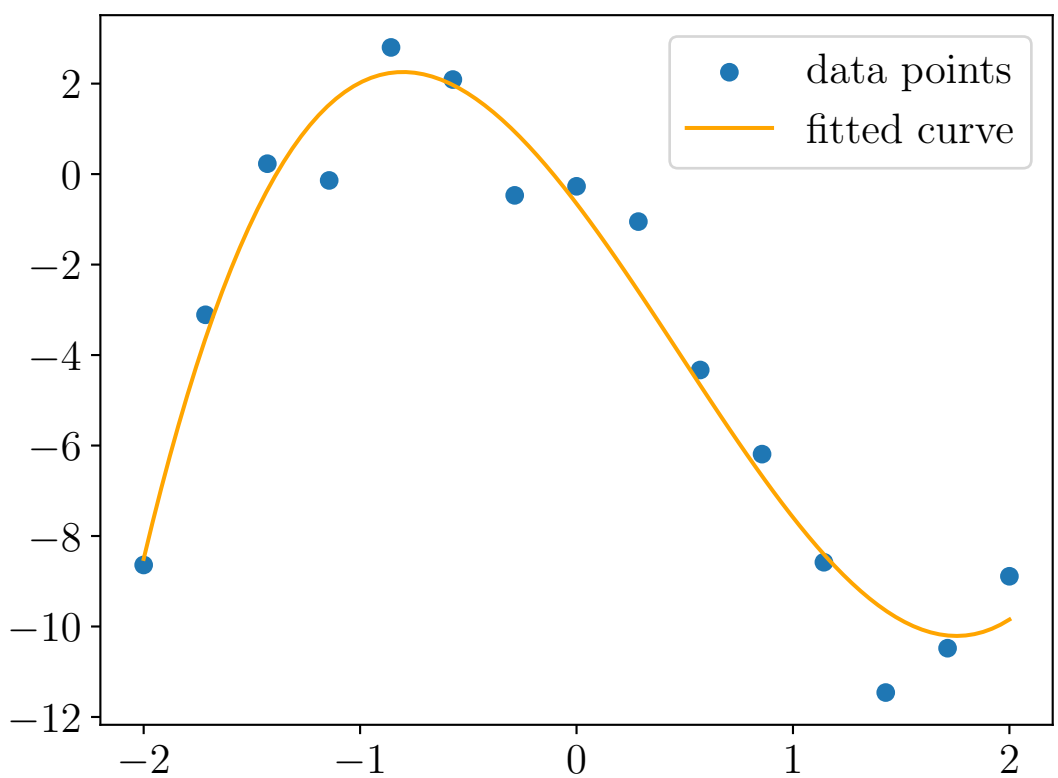

Figure 3: Our cubic polynomial fitted from least squares approximately matches the given data points.

The Matlab code is as follows:

```
m = 4; % degree of polynomial
n = 15; % number of data points

% define your data
a = linspace(-2,2,15);
b = [-8.64, -3.11, 0.23, -0.14, 2.80, 2.09, -0.47, -0.27, -1.05,
   -4.33, -6.19, -8.58, -11.46, -10.48, -8.89];
b = transpose(b);

% plot the data first
tiledlayout(3,1)
nexttile
scatter(a, b)
hold on;

% generate the A matrix:
% A = [1 a_1 a_1^2 a_1^3;
%      1 a_2 a_2^2 a_2^3;
%      ................;
%      1 a_n a_n^2 a_n^3]
A = [ones(size(a)); a; a.^2; a.^3];
A = transpose(A);


% Set initial guess, step size, maximum iterations, and tolerance
cc0 = zeros(m, 1);
alpha = 4*1e-3; % step size
max_iter = 1000;
tol = 1e-6;

c = cc0;
f_hist = zeros(max_iter);
fonc_hist = zeros(max_iter);
for i = 1:max_iter
    grad = transpose(A) * (A * c - b);
    fonc_curr = norm(grad);
    fonc_hist(i) = fonc_curr;
    f_hist(i) = .5 * norm(A * c - b)^2;
    if fonc_curr < tol
        break
    end
    c = c - alpha * grad;
```

```
end

cc_opt = c;

% Print the estimated parameter vector
cc_opt

xv = linspace(-2,2,100);
yv = cc_opt(1) + cc_opt(2)*xv + cc_opt(3)*xv.^2 + cc_opt(4)*xv.^3;


% plot results
plot(xv,yv)
hold off;
ylabel('y')
xlabel('x')

nexttile
semilogy(fonc_hist)
ylabel('gradient norm')
xlabel('iterations')

nexttile
semilogy(f_hist)
ylabel('obj val')
xlabel('iterations')


% Newton direction can be found in one iteration
cc_newton = -inv(transpose(A) * A) * transpose(A) * b;
cc_newton
```

The Python code is as follows:

```
import numpy as np
import matplotlib.pyplot as plt

m = 4 # degree of polynomial
n = 15 # number of data points

# define your data
a = np.linspace(-2,2,15)
b = np.array([-8.64, -3.11, 0.23, -0.14, 2.80, 2.09, -0.47, -0.27,
```

```
    -1.05, -4.33, -6.19, -8.58, -11.46, -10.48, -8.89])
b = b.reshape(-1,1)

# plot the data first
plt.plot(a, b, 'o')

# generate the A matrix:
# A = [1 a_1 a_1^2 a_1^3;
#      1 a_2 a_2^2 a_2^3;
#      ................;
#      1 a_n a_n^2 a_n^3]
A = np.array([np.ones_like(a), a, a**2, a**3])
A = A.transpose()

# Define least squares objective function
def f(x):
    return np.linalg.norm(A @ x - b)**2 / 2

# Define gradient of least squares objective function
def grad_f(x):
    return A.T @ (A @ x - b)

# Define steepest descent algorithm
def steepest_descent(c0, alpha, max_iter, tol):
    c = c0
    for i in range(max_iter):
        grad = grad_f(c)
        if np.linalg.norm(grad) < tol:
            break
        c -= alpha * grad
    return c

# Set initial guess, step size, maximum iterations, and tolerance
cc0 = np.zeros((m, 1))
alpha = 4*1e-3 # step size
max_iter = 1000
tol = 1e-6

# Steepest descent algorithm
c = cc0
fonc_hist = []
f_hist = []
```

```
for i in range(max_iter):
    grad = grad_f(c)
    fonc_curr = np.linalg.norm(grad)
    fonc_hist.append(fonc_curr)
    f_hist.append(f(c))
    if fonc_curr < tol:
        break
    c -= alpha * grad

cc_opt = c

# Print the estimated parameter vector
print('Estimated parameter vector:')
print(cc_opt)

xv = np.linspace(-2,2,100)
yv = cc_opt[0] + cc_opt[1]*xv + cc_opt[2]*xv**2 + cc_opt[3]*xv**3


# plot results
plt.plot(xv,yv)
plt.show()

iters = range(i+1)
plt.plot(iters, fonc_hist)
plt.yscale('log')
plt.show()

plt.plot(iters, f_hist)
plt.show()


# Newton direction can be found in one iteration
cc_newton = -np.linalg.inv(A.T @ A) @ A.T @ b
print(cc_newton)
```